计算机学院

数据安全实验报告

实验 2: 基于 libsnark 的 zkSNARK 零知识证明实验

姓名：王旭

学号：2312166

专业：计算机科学与技术

2026 年 4 月 16 日

目录

1 实验名称 **2**

2 实验目的 **2**

3 实验原理 **2**

3.1 非交互式零知识证明（NIZK）介绍与原理 2

3.1.1 zkSNARK 技术介绍 2

3.1.2 数学转换流程 2

3.2 libsnark 开发框架介绍 3

3.3 libsnark 工作原理 3

3.4 变量管理与 Gadget 电路组件 4

3.5 框架子模块（Submodule）与底层依赖 4

4 实验环境配置流程 **4**

4.1 基础系统环境与依赖安装 5

4.2 ZIP 源码获取与子模块重组 5

4.3 库核心编译与问题解决汇总 6

4.4 实验业务部署与环境验证 7

5 实验进行流程 **7**

5.1 实现流程与思路 7

5.2 具体实现代码 8

5.2.1 common.hpp (公共电路构造与配置) 8

5.2.2 mysetup.cpp (第一阶段：生成证明密钥和验证密钥) 9

5.2.3 myprove.cpp (第二阶段：提交数据与出具证明) 10

5.2.4 myverify.cpp (第三阶段：外部校验与接受证明) 11

5.3 代码运行结果与分析 12

5.3.1 测试脚本运行结果 12

5.3.2 手动运行结果 14

5.3.3 运行结果分析 14

6 实验总结 **15**

6.1 主要工作 15

6.2 实验心得体会 15

7 github 仓库 **16**

1 实验名称

基于 libsnark 的 zkSNARK 零知识证明实验

2 实验目的

参考教材实验 3.1，假设 Alice 希望证明自己知道如下方程的解 $x^3 + x + 5 = out$,。其中 out 是大家都知道的一个数，这里假设 out 为 35，而 $x = 3$ 就是方程的解，请实现代码完成明生成和证明的验证。

3 实验原理

3.1 非交互式零知识证明（NIZK）介绍与原理

零知识证明（Zero-Knowledge Proof, ZKP）是一种由证明者（Prover）向验证者（Verifier）证明某项论断为真，却不泄露除“该论断为真”之外任何有用信息的密码学协议。该协议必须满足三大基本性质：

1. **完整性（Completeness）**：若论断真实，诚实的证明者总能说服诚实的验证者。
2. **可靠性（Soundness）**：若论断虚假，伪造的证明者以压倒性的概率无法欺骗验证者。
3. **零知识性（Zero-Knowledge）**：验证过程中，验证者无法获取到证明者的任何隐秘信息（即 Witness，见证值）。

3.1.1 zkSNARK 技术介绍

zkSNARK（Zero-Knowledge Succinct Non-Interactive Argument of Knowledge）是一种广为应用的零知识证明技术，其突出特性在于：

- **简洁性（Succinct）**：生成的证明（Proof）体积极小（通常仅为几百字节级别），并且无论原始计算逻辑多复杂，验证者验证该证明所需的时间都极短且基本保持常数级别。
- **非交互式（Non-Interactive）**：有别于早期的多次挑战-响应模型，zkSNARK 利用公共参考串（Common Reference String, CRS）等机制，只需证明者生成并发送一次证明，验证者即可独立完成验证，无需双方反复通信。

3.1.2 数学转换流程

将普通计算逻辑转化为零知识证明的底层运算，通常经历以下多步转换：

1. **算术电路（Arithmetic Circuit）构建**：将原始逻辑转化为包含乘法门和加法门的计算图。
2. **一阶约束系统（R1CS, Rank-1 Constraint System）转换**：算术电路中的运算被编码为满足 $A \cdot B = C$ 形式的一系列线性组合约束方程。
3. **二次算术程序（QAP, Quadratic Arithmetic Programs）归约**：通过拉格朗日插值等方法，将大量 R1CS 方程转换为多项式问题，使约束条件的验证可以通过检验单一多项式在特定点的值来完成。

4. **配对与验证**：最终结合同态隐藏和椭圆曲线双线性配对计算，在不暴露真实数据的情况下评估约束是否成立，并由验证算法给出结果。

3.2 libsnark 开发框架介绍

`libsnark` 是由 SCIPR Lab 等学术界主导维护的开源 C++ zkSNARK 密码学框架。由于其较好的安全性和在实际区块链项目（如 Zcash）中的成功落地，在业内具有很高的研究与应用价值。

libsnark 的设计注重**模块化**与 **C++ 泛型（Template）编程**的支持。开发者可以通过框架提供的应用接口编写电路的逻辑约束，框架会负责调用底层密码库完成具体的代数推导、参数插值和密钥生成工作：

- 框架支持配置不同的安全曲线后端，例如 BN128（即 alt_bn128 曲线）、BLS12-381 等素数阶循环群。
- 框架复用了底层的代数库与计算方法，内置了 Groth16、PGHR13 等多种 zkSNARK 协议，同时在底层提供了不同约束表达（从 BACS 到 R1CS，再到 QAP 等）的转换接口。这不仅保障了框架的理论拓展性，也方便开发者进行相关的性能优化定制。

3.3 libsnark 工作原理

在确定证明协议（例如 `r1cs_gg_ppzksnark`）与底层曲线后，libsnark 会按照以下三个关键阶段执行整个验证流程：

1. **初始化阶段（Setup）**：
   - **输入**：针对当前计算命题定义的 R1CS 约束系统对象（不包含变量具体赋值）。
   - **执行过程**：框架内部会采样随机数种子（通常称为 Toxic Waste），基于 QAP 相关系数计算全局系统参数。
   - **输出结果**：生成**证明密钥（Proving Key, PK）**和**验证密钥（Verification Key, VK）**。其中，PK 的数据量通常与逻辑电路的规模成正比，而 VK 的数据量相对很小。此阶段使用的底层随机数种子在生成参数后必须被销毁，否则验证系统可能被伪造。
2. **证明生成阶段（Prove）**：
   - **输入**：公开输入数据（本实验中的 $out = 35$）、私密的见证数据（Witness，即 $x = 3$）以及前一步生成的 PK。
   - **执行过程**：程序将输入数据与 Witness 分别赋值给电路中的变量对象。在约束成立的前提下，结合 PK 对数据进行对应的多项式求值等密码学组装操作，将计算过程压缩处理。
   - **输出结果**：生成简洁的**证明数据（Proof）**。
3. **验证阶段（Verify）**：
   - **输入**：初始化阶段生成的 VK、公开输入数据以及证明者提供的 Proof。
   - **执行过程**：验证者利用 VK 和公开输入，通过双线性配对（Bilinear Pairing，类似于校验 $e(A, B) = e(C, D)$ 的形式）对 Proof 进行验证。在此过程中不涉及也不泄露任何有关私密见证数据的内容，如果通过验证，则表明证明者确实掌握满足整个约束条件的原始解。

3.4 变量管理与 Gadget 电路组件

为降低手工构建由于大量 R1CS 约束方程所带来的编写门槛，libsnark 的 `gadgetlib1` 开发包采用了一种模块化组装电路的设计理念：

- **protoboard（面包板）**：负责管理电路的各种结构数据以及运行状态。无论是变量空间的分配申请、代数传值的暂存，还是各个逻辑门 R1CS 约束关系的添加保存，均需通过 `protoboard` 对象进行统一调度。
- **pb_variable（变量）与 pb_variable_array（变量数组）**：这些类并不直接存储普通的数值数据，而是用来代指该算术网络内对应连线或变量的引脚编号。开发者可以定义其为系统的公开输入（Primary Input）或系统内的中间私密变量（Auxiliary Input）。
- **Gadget（可复用组件）**：一些常规但复杂的算法（例如哈希运算 SHA256、数字大小比较器 Comparator 等）通常需要数十甚至数百个约束方程。libsnark 支持将其统一封装成 Gadget C++ 继承体系组件。开发者只需如同调用模块一样向其传入联接的 `pb_variable` 参数即可完成接入，从而大幅提升了代码可读性与电路设计复用率。

3.5 框架子模块（Submodule）与底层依赖

出于密码学安全与运算性能的要求，libsnark 在底层高度依赖多个开源的外部基础实现库。在代码组织上，它利用了 Git 的 `submodule`（子模块）机制，将其引入主模块的 `depends/` 配置目录之下，联合编译。核心的依赖包括：

- **libff**：核心算术运算引擎。专注于提供高精度底层素数域框架与密码学原语支撑，处理运算与椭圆曲线的基础逻辑大都依赖此底座。
- **libfqfft**：专注于提供有限域中的快速傅里叶变换（FFT）支持。其运用可以将 QAP 评估阶段有关多项式的运算时间复杂度较好地由 $O(n^2)$ 优化至 $O(n \log n)$ 表内，对大规格电路生成证明的性能保障影响非常大。
- **ate-pairing 与 xbyak**：负责底层的曲线配对运算加速机制支撑。`ate-pairing` 通过 `xbyak` 即时汇编引擎将处理映射为高效的 C++ 级底层指令，能够对最终响应配对和检验运算起实质优化。
- **gtest 与 libsnark-supercop**：提供开发过程中的核心单元测试组件与部分基础常用加密原语算法的支持集。

正因不同模块工程相互分离的现状特性，编译源码必须使用正确指定的兼容版本才可运作。在正常基于网络 clone 的情况下 `.gitmodules` 等文件可负责绑定，但假设开发所用的是直接打包的 zip 拉取等偏离状态，相关的子文件往往并未真正进入到下载序列。这就使得手动搭建该环境会更容易产生如编译器缺失特定 API、找不到库引用而未定义的抛错等环境报错教训。

4 实验环境配置流程

本次实验将实验代码框架按照功能划分为三大核心模块：`deps`（存放原框架代码及底层子模块依赖）、`scripts`（存放自动化构建与运行脚本）以及 `src`（存放核心的 C++ 零知识证明原始业务代码）。

4.1 基础系统环境与依赖安装

在 Linux（如 Ubuntu）系统环境下，构建 libsnark 依赖于 C++ 编译工具链以及多个数学底层支持库。首先需要通过系统包管理器安装相关的核心依赖环境：

```
sudo apt-get update
sudo apt-get install build-essential cmake git libgmp3-dev libprocps-dev python3
    markdown libboost-program-options-dev libssl-dev python3-pkgconfig
```

其中，

- `libgmp3-dev` (GMP) 提供多精度大数算术库，是处理加密算法中大质数运算的基础
- `libboost-program-options-dev` 等用于提供基础的 C++ 程序开发支持。

4.2 ZIP 源码获取与子模块重组

(1) 源码获取的网络困境与 ZIP 方式

在理想网络环境下，获取包含完整依赖的 libsnark 源码应该通过执行 `git clone --recursive` 直接拉取主工程及所有子模块。但在我的实际配置过程中，由于网络环境受限等多方面原因 (尝试挂了梯子也不行)，`git clone` 频繁超时中断，一直无法完整获取源代码框架。因此，我采取了在 Windows 系统上手动登录 GitHub 下载 libsnark 源码 ZIP 压缩包的方式，随后解压并移入到 Linux 实验环境的 `deps/libsnark` 目录中。

(2) ZIP 压缩包带来的问题：子模块丢失

这种手动下载 ZIP 包的方式虽然规避了主工程的克隆超时问题，但对于该 libsnark 代码框架而言，带来了一个很大的问题：ZIP 包仅仅包含主分支的静态文件，**而完全不包含** `.git` 版本控制记录，使得框架下属极为关键的 Submodule（子模块，如基础算术引擎 libff）目录全都是空文件夹。直接对这类残缺的 libsnark 源码框架进行 `cmake` 必然导致比如“未找到 libff”、“缺失 gtest”等各类的编译报错。

(3) 子模块的手动获取与组装方式

为了解决离线包缺失子模块的问题，我们在 `deps/_tmp_submodules` 目录下采取了手动定位并重组所有依赖模块的方法：

- **定位版本**：通过查阅主项目根目录下的 `.gitmodules` 文件，明确 `libff`，`libfqfft`，`xbyak`，`ate-pairing`，`gtest`，`libsnark-supercop` 等核心子模块应当绑定的确切分支或 Commit 版本。

根目录下的.gitmodules 文件

```
[submodule "depends/gtest"]
    path = depends/gtest
    url = https://github.com/google/googletest.git
[submodule "depends/ate-pairing"]
    path = depends/ate-pairing
    url = https://github.com/herumi/ate-pairing.git
[submodule "depends/xbyak"]
    path = depends/xbyak
    url = https://github.com/herumi/xbyak.git
```

```
[submodule "depends/libsnark-supercop"]
    path = depends/libsnark-supercop
    url = https://github.com/mbbarbosa/libsnark-supercop.git
[submodule "depends/libff"]
    path = depends/libff
    url = https://github.com/scipr-lab/libff.git
[submodule "depends/libfqfft"]
    path = depends/libfqfft
    url = https://github.com/scipr-lab/libfqfft.git
```

- **定向下载**：我们利用了 `curl` 等网络下载工具，在线直接拉取各子模块特定版本的 ZIP 压缩包并解压。随后去除解压产生的冗余分支名后缀，将它们重命名并移动到 `deps/libsnark/depends/` 对应目录下。结合上述 `.gitmodules` 里的配置映射关系，以获取 `libff` 和 `gtest` 为例，具体的操作步骤如下：

```
# 下载并配置 libff 子模块
curl -L -o libff.zip https://github.com/scipr-lab/libff/archive/refs/heads/master.zip
unzip libff.zip
mv libff-master deps/libsnark/depends/libff

# 下载并配置 gtest (googletest) 子模块
curl -L -o gtest.zip
    https://github.com/google/googletest/archive/refs/heads/master.zip
unzip gtest.zip
mv googletest-master deps/libsnark/depends/gtest
```

  (其它的比如 `libfqfft`、`xbyak` 等模块以此类推，均需要严格对齐对应目标 `path` 。)

- **处理嵌套依赖**：需要留意的是，子模块之间也存在嵌套依赖。例如 `libff` 本身除了提供底层素数域框架原语外，其目录结构下同样存在它自己的内部依赖，在重组时必须逐层补齐，以保证后续框架功能体系的完整发挥。

4.3 库核心编译与问题解决汇总

在确保主工程及其所有的各级子模块代码均已完整放置到对应位置后，进入 CMake 源码外构建（Out-of-source build）流程：

CMake 指令

```
mkdir build && cd build
cmake ..
make -j$(nproc)
```

配置编译期遇到的问题与解决总结：

1. **CMake 失败或报错未找到目标库 / include 不存在**：通常是由于子模块没有被放在正确层级，或者目录名包含了额外的 `-src` 或分支名后缀而未被准确更名（如 `libff-src` 未重命名为 `libff`），导致主 `CMakeLists.txt` 的相对路径扫描失败。解决办法是严格核对好我们主工程下的 `deps/libsnark/depends` 目录下的文件夹命名。

2. **特定 API 找不到引用的编译器抛错**：不同子模块之间版本极度敏感，如果下载了主分支最新版而非 libsnark 当时代码锁定的版本，由于头文件变动频繁，会引发大量的 C++ 编译层不兼容报错。解决办法是必须严格对照原框架要求的具体节点获取对应子模块的源码包。

修复全部依赖后编译完成，系统会在 `build/lib` 目录下生成 `libsnark.a` 等核心静态库，然后在 `build/depends/libff/lib` 下生成 `libff.a` 库文件，这些组件构成了后续开发的基础库支持。

4.4 实验业务部署与环境验证

为了验证我们的实验环境是否搭建成功并可被外部开发调用，我们基于分离的思路构建了我们实验文件的层级结构：

```
zk_lab_exp31/
    deps/
        libsnark/        # 处理完毕、含完整子模块的libsnark底层框架代码
    scripts/
        build_and_run.sh # 自动化构建与正反向用例测试脚本
    src/
        common.hpp       # 抽离的辅助函数，主要负责系统输出的净化处理
        mysetup.cpp      # 初始化阶段可执行程序：用于生成 pk 与 vk
        myprove.cpp      # 证明生成阶段程序：引入公开数据与加密见证
        myverify.cpp     # 验证阶段程序：基于双线性配对的最终校验
    CMakeLists.txt       # 配置头文件引用搜索路径及 libsnark、libff
    链接设置的构建脚本
```

实验环境的成功验证标志：

为了检验我们本次实验环境是否真正配置完成，我们需要看 `CMakeLists.txt` 能否被正确解析，并成功在 libsnark 代码框架的各节点上编译 C++ 源码。

当我们通过编译指令成功将 `mysetup`，`myprove`，`myverify` 链接为独立生成的可执行文件，且终端没有报出诸如 `undefined reference to 'libff::xxx'` 等底层的链接错误时，就证明 libsnark 的核心静态库与头文件节点已经链接好了我们的实验框架，我说明我们成功解决了残缺库依赖的问题，配置好了实验环境。

5 实验进行流程

5.1 实现流程与思路

本实验的核心目标是利用零知识证明向验证方证明“我知道方程 $x^3 + x + 5 = 35$ 的解”，并且全过程无需透露 x 的具体数值。在实际密码学实现中，高级数学方程无法被 `libsnark` 自带的运算机制直接接收使用。因此，我们首先需要将复杂的代数方程式转化为基本的加法与乘法门计算网络，即转化为符合 $A \times B = C$ 单一组合形式的 R1CS（一阶约束系统）约束条目。

将方程 $x^3 + x + 5 = out$（其中明确预设公开输入 $out = 35$）拆解与转换为了以下关联的四个计算步骤：

1. $x \times x = w_1$ （算得 x^2）

2. $w_1 \times x = w_2$ （算得 x^3）

3. $(w_2 + x) \times 1 = w_3$ （算得 $x^3 + x$）

4. $(w_3 + 5) \times 1 = out$ （最终得到 $x^3 + x + 5$ 的总值关系，并结合验证标识变量 out 并集）

确定上述约束关系后，通过 C++ 代码配合使用 `libsnark` 的核心管理载体 `protoboard`（面包板）来实现落地：

- **声明变量**：定义需要的计算通信节点，包括 5 个运算变量 `out`，`x`，`w_1`，`w_2`，和 `w_3`。
- **注册约束**：依次利用 `add_r1cs_constraint` 方法，将预先分析构造的连环方程式（R1CS）添加至总体逻辑拓扑中。
- **代入计算**：当系统获取到用户传入的有效解答数值（即 Witness 证据值）后，会分别运算并赋值给整个电路的其余关联引脚；随后移交给框架进行最后的计算，完成完整的密码学生成。

5.2 具体实现代码

下面我们来分析与我们实施思路对应的四个核心实现模块。文件存放在我们目录的 `zk_lab_exp31/src/` 中。

5.2.1 common.hpp (公共电路构造与配置)

```cpp
#pragma once

#include <cstdlib>
#include <cstdio>
#include <fcntl.h>
#include <unistd.h>

#include <libff/common/profiling.hpp>
#include <libsnark/common/default_types/r1cs_gg_ppzksnark_pp.hpp>
#include <libsnark/gadgetlib1/pb_variable.hpp>
#include <libsnark/relations/constraint_satisfaction_problems/r1cs/r1cs.hpp>

using namespace libsnark;
typedef libff::Fr<default_r1cs_gg_ppzksnark_pp> FieldT;

/* --------- (输出净化辅助控件StdoutSilencer) --------- */
// 主要目的为静默 libsnark C 层的底层大量 printf 控制台输出
/* ------------------------------------ */

// 构建含有业务组建与 R1CS 限定约束关系的通用底层模型（面包板）
inline protoboard<FieldT> build_protoboard(const int *secret) {
  // 1. 初始化底层椭圆曲线的公开参数配置组件
  default_r1cs_gg_ppzksnark_pp::init_public_params();
  protoboard<FieldT> pb;

  // 2. 声明当前电路网路操作所需要定义的基础关联计算标识符
  pb_variable<FieldT> out;
  pb_variable<FieldT> x;
```

```
  pb_variable<FieldT> w_1;
  pb_variable<FieldT> w_2;
  pb_variable<FieldT> w_3;

  // 3. 为各项声明变量实体分配资源空间并实施命名操作
  out.allocate(pb, "out");
  x.allocate(pb, "x");
  w_1.allocate(pb, "w_1");
  w_2.allocate(pb, "w_2");
  w_3.allocate(pb, "w_3");

  // 4. 设置主要的公开节点输入量大小。由于率先进行分配操作的为 out 实体,
  // 结合该定义后, out 将直接用作系统公开的数据检验入口。
  pb.set_input_sizes(1);
  pb.val(out) = 35; // 设置本计算多项式的公开输出检验标准必须等于 35

  // 5. 依次并完整补充一阶加乘约束操作方程 (维持 A * B = C的格式体)
  //  x * x = w_1
  pb.add_r1cs_constraint(r1cs_constraint<FieldT>(x, x, w_1));
  //  w_1 * x = w_2
  pb.add_r1cs_constraint(r1cs_constraint<FieldT>(w_1, x, w_2));
  //  (w_2 + x) * 1 = w_3
  pb.add_r1cs_constraint(r1cs_constraint<FieldT>(w_2 + x, 1, w_3));
  //  (w_3 + 5) * 1 = out
  pb.add_r1cs_constraint(r1cs_constraint<FieldT>(w_3 + 5, 1, out));

  // 6. 计算流程导入操作: 假设函数目前接收到了非空传参(验证阶段提供), 则依次为电路管
  // 线赋予运算后推导的真实代数结果。
  if (secret != nullptr) {
    pb.val(x) = secret[0];
    pb.val(w_1) = secret[1];
    pb.val(w_2) = secret[2];
    pb.val(w_3) = secret[3];
  }

  return pb;
}
```

`common.hpp` 中所设定的 `build_protoboard` 功能作为本项工作的约束处理中枢，基于变量节点挂载声明，为随后三阶段计算搭建了通用可复用的电路基底构建模块。

5.2.2 mysetup.cpp (第一阶段：生成证明密钥和验证密钥)

```
#include <fstream>
#include <iostream>
#include <libsnark/zk_proof_systems/ppzksnark/r1cs_gg_ppzksnark/r1cs_gg_ppzksnark.hpp>
#include "common.hpp"

```

```
int main() {
  configure_libsnark_profiling_from_env();

  // 1. 调用提前设置的封装逻辑，只提取不含有运算 Witness 的框架构成约束拓扑。
  protoboard<FieldT> pb = build_protoboard(nullptr);
  const r1cs_constraint_system<FieldT> constraint_system = pb.get_constraint_system();

  // 2. 利用提取拓扑传入密码生成库生成证明密钥和验证密钥 PK与 VK 。
  const r1cs_gg_ppzksnark_keypair<default_r1cs_gg_ppzksnark_pp> keypair = [&]() {
    StdoutSilencer silencer(!zk_profiling_enabled());
    return
        r1cs_gg_ppzksnark_generator<default_r1cs_gg_ppzksnark_pp>(constraint_system);
  }();

  // 3. 产出体积较高的证明公钥 PK 并向本地路径做写入留档
  std::fstream pk("pk.raw", std::ios_base::out);
  pk << keypair.pk;
  pk.close();

  // 4. 将验证器专用的验证密钥 VK 文件也序列化写入本地
  std::fstream vk("vk.raw", std::ios_base::out);
  vk << keypair.vk;
  vk.close();

  std::cout << "初始化完成：已生成 pk.raw 和 vk.raw" << std::endl;
  return 0;
}
```

在此程序中我们获取到没有真值变量代入的空白面包板结构。系统计算后产生证明密钥和验证密钥 `pk` 与 `vk`。运算所需的一次性乱序的随机变量资源会被废弃，以保持系统有效性。

5.2.3 myprove.cpp (第二阶段：提交数据与出具证明)

```
#include <fstream>
#include <iostream>
#include <libsnark/zk_proof_systems/ppzksnark/r1cs_gg_ppzksnark/r1cs_gg_ppzksnark.hpp>
#include "common.hpp"

int main() {
  configure_libsnark_profiling_from_env();

  int x = 0;
  std::cout << "请输入私密值 x: " << std::flush;
  std::cin >> x;

  // 补充输出控制：如果由脚本重定向自动执行，将读取变量x进行同步显现以便运行确认
  if (!::isatty(STDIN_FILENO)) {
    std::cout << x << std::endl;
```

```
  } else {
    std::cout << std::endl;
  }

  // 1. 获取输入的 x 值，并计算相关的各个中间变量值
  int secret[4];
  secret[0] = x;
  secret[1] = x * x;
  secret[2] = x * x * x;
  secret[3] = secret[2] + x;

  // 2. 将包含这些实参真值的数组传入面包板中，完成各节点的赋值
  protoboard<FieldT> pb = build_protoboard(secret);

  // 3. 从本地文件中读取上一阶段生成的证明密钥 (PK)
  std::fstream f_pk("pk.raw", std::ios_base::in);
  r1cs_gg_ppzksnark_proving_key<libff::default_ec_pp> pk;
  f_pk >> pk;
  f_pk.close();

  // 4. 调用 Prover 算法，结合电路约束和 PK 生成最后的零知识证明 (Proof)
  const r1cs_gg_ppzksnark_proof<default_r1cs_gg_ppzksnark_pp> proof = [&]() {
    StdoutSilencer silencer(!zk_profiling_enabled());
    return r1cs_gg_ppzksnark_prover<default_r1cs_gg_ppzksnark_pp>(pk,
        pb.primary_input(), pb.auxiliary_input());
  }();

  // 5. 将生成的证明数据做本地保存，供验证阶段使用
  std::fstream pr("proof.raw", std::ios_base::out);
  pr << proof;
  pr.close();

  std::cout << "证明生成完成: proof.raw" << std::endl;
  return 0;
}
```

该部分程序作为证明者（Prover），其核心任务是接收用户输入的私密值 `x`。程序计算出所需各个中间变量并代入电路，然后利用第一步生成的证明密钥（PK），计算出一个体积极小的零知识证明（Proof）文件并保存至本地。

5.2.4 myverify.cpp (第三阶段：外部校验与接受证明)

```
#include <fstream>
#include <iostream>
#include <libsnark/zk_proof_systems/ppzksnark/r1cs_gg_ppzksnark/r1cs_gg_ppzksnark.hpp>
#include "common.hpp"

int main() {
```

```
  configure_libsnark_profiling_from_env();

  // 1. 验证器只需要电路的网络结构，不需要具体私密输入，所以传入 nullptr
      构建空约束拓扑
  protoboard<FieldT> pb = build_protoboard(nullptr);

  // 2. 从本地读取用于验证的密钥 (VK)
  std::fstream f_vk("vk.raw", std::ios_base::in);
  r1cs_gg_ppzksnark_verification_key<libff::default_ec_pp> vk;
  f_vk >> vk;
  f_vk.close();

  // 3. 读取证明者发送过来的零知识证明文件 (Proof)
  std::fstream f_proof("proof.raw", std::ios_base::in);
  r1cs_gg_ppzksnark_proof<libff::default_ec_pp> proof;
  f_proof >> proof;
  f_proof.close();

  // 4. 调用验证接口，根据 VK、公开输入以及 Proof 进行对等评估比对，返回结果
  bool verified = false;
  {
    StdoutSilencer silencer(!zk_profiling_enabled());
    verified = r1cs_gg_ppzksnark_verifier_strong_IC<default_r1cs_gg_ppzksnark_pp>(vk,
        pb.primary_input(), proof);
  }

  std::cout << "验证结果: " << verified << std::endl;
  return 0;
}
```

该部分程序作为验证者 (Verifier)，在不知道秘密值 `x` 的前提下，只凭借轻量级的验证密钥 (VK)、公开的约束要求以及证明者发来的 Proof，就能通过底层比对算法来确定证明者是否真的知晓方程解。验证通过则判定成功输出 1，否则输出 0。

5.3 代码运行结果与分析

5.3.1 测试脚本运行结果

我们在项目主目录执行了 `scripts/build_and_run.sh` 脚本来进行自动传参测试，运行同时包含了正例 $x = 3$ 与反向测试用例 $x = 2$ 的具体实验执行与验证情况。脚本内容如下：

build_and_run.sh 测试脚本

```
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
BUILD_DIR="$ROOT_DIR/build"
PREFIX_DIR="$ROOT_DIR/deps/install"

```

```
cmake -S "$ROOT_DIR" -B "$BUILD_DIR" -DLIBSNARK_PREFIX="$PREFIX_DIR"
cmake --build "$BUILD_DIR" -j"$(nproc)"

cd "$BUILD_DIR"
./mysetup

echo "========== 正例: x=3 (期望 验证结果: 1) =========="
printf "3\n" | ./myprove
./myverify

echo "========== 反例: x=2 (期望 验证结果: 0) =========="
printf "2\n" | ./myprove
./myverify
```

输出内容参考如下:

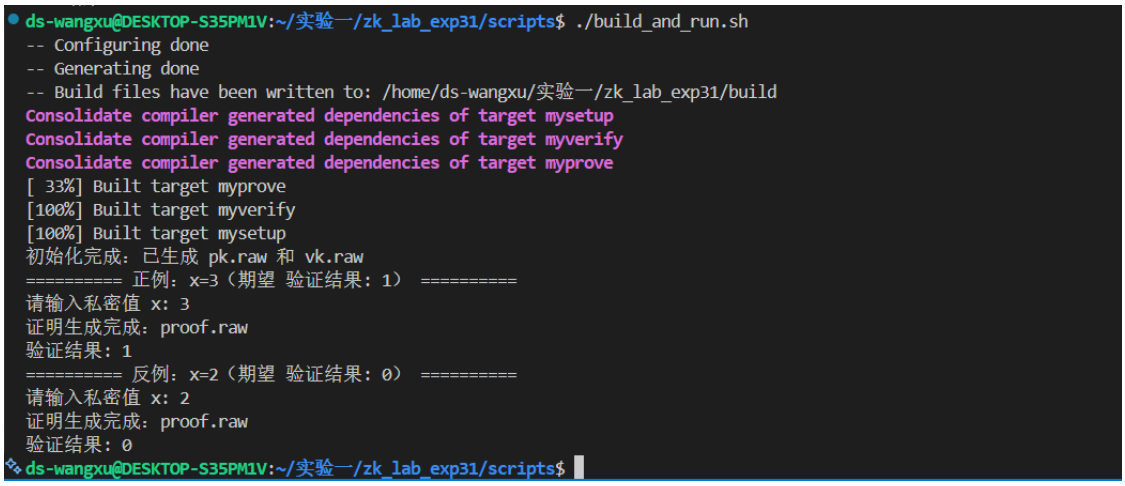

图 5.1: 测试脚本运行输出结果

测试脚本输出结果

```
ds-wangxu@DESKTOP-S35PM1V:~/实验一/zk_lab_exp31/scripts$ ./build_and_run.sh
-- Configuring done
-- Generating done
-- Build files have been written to: /home/ds-wangxu/实验一/zk_lab_exp31/build
Consolidate compiler generated dependencies of target mysetup
Consolidate compiler generated dependencies of target myprove
Consolidate compiler generated dependencies of target myverify
[ 33%] Building CXX object CMakeFiles/myprove.dir/src/myprove.cpp.o
[ 33%] Building CXX object CMakeFiles/mysetup.dir/src/mysetup.cpp.o
[ 50%] Building CXX object CMakeFiles/myverify.dir/src/myverify.cpp.o
[ 66%] Linking CXX executable myverify
[ 66%] Built target myverify
[ 83%] Linking CXX executable myprove
[100%] Linking CXX executable mysetup
[100%] Built target myprove
[100%] Built target mysetup
```

```
初始化完成：已生成 pk.raw 和 vk.raw
========== 正例：x=3（期望 验证结果：1） ==========
请输入私密值 x：3
证明生成完成：proof.raw
验证结果：1
========== 反例：x=2（期望 验证结果：0） ==========
请输入私密值 x：2
证明生成完成：proof.raw
验证结果：0
```

5.3.2 手动运行结果

我们在 build 目录下进行如下的手动运行测试：

手动测试指令

```
./mysetup                    # 初始化
./myprove                    # 证明阶段(x=3)
./myverify                   # 验证阶段
./myprove                    # 证明阶段(x=2)
./myverify                   # 验证阶段
```

运行结果如下所示：

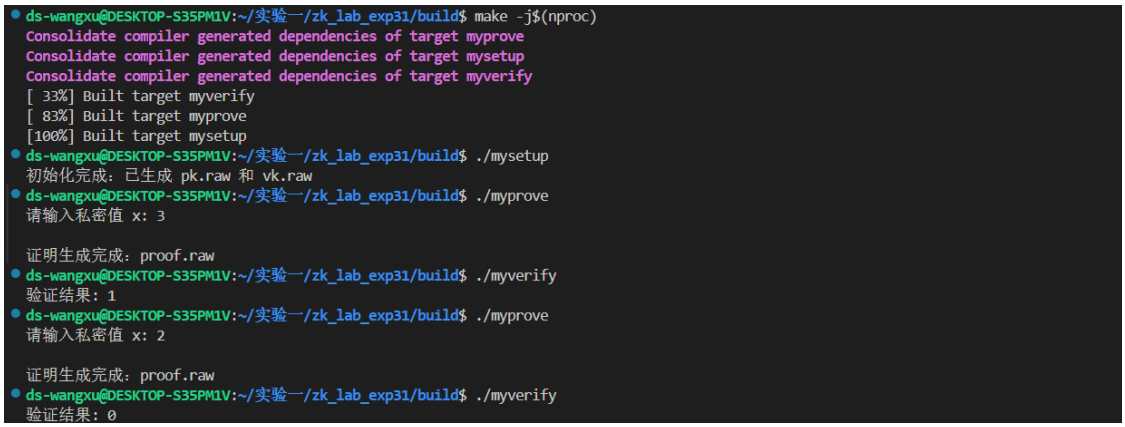

图 5.2: 手动测试运行输出结果

5.3.3 运行结果分析

对运行结果的具体分析如下：

1. **网络初始化与密钥生成：**
   - `mysetup` 进程顺利基于 CMake 构建了网络结构，初始化相关底层组件计算。
   - 产生了对应的核心基础文件 `pk.raw` 和 `vk.raw` 模型。
   - 此步骤的成功标志着验证流程全局所需的环境已搭建就绪。
2. **正向证明与验证（`x=3`）：**

- 当用户输入正确的私密值 `3` 后（显然 $3^3+3+5=35$ 符合方程），参数在内部各约束点能够正确闭合匹配，并在 Prover 阶段合法组装，输出证明数据 `proof.raw`。
- 验证端的 `myverify` 程序提取 `proof.raw` 时，系统底层配对计算确认符合预设规律，等式成立。
- 终端随即输出正反馈结果 `验证结果: 1`。这展示了系统中“真实的知情者必能通过验证”的**完整性（Completeness）**表现。

3. **反例阻挡测试（`x=2`）**：
   - 当传入错误的值 `2` 时（因为 $2^3+2+5=15\neq 35$，不满足系统要求的输出验证点），电路约束关系无法对应和闭合。
   - 程序强行算出的残余数据被生成了一份不满足要求、发生错误的记录证明发往接收方。
   - 验证端 `myverify` 在评估这个错误证明时由于参数匹配校验失败被直接阻滞，终端当即输出拦截结果 `验证结果: 0`。
   - 这种测试结果反映出虚假的数据确实无法通过系统检查，满足了协议具备防伪造机制的**可靠性（Soundness）**功能。

6 实验总结

6.1 主要工作

本次实验我们围绕 libsnark 代码框架展开，从零开始，逐步完成了 libsnark 实验环境配置、面包板的算术电路编写、setup 密钥生成与初始化、prove 证明与 verify 验证，最后通过 x 的正反例测试验证了系统的完整性与可靠性。通过测试脚本，分别针对正例（$x=3$）与反例（$x=2$）进行了验证。只有持有真实私密数据的一端才能生成合法证明以通过检查，实验结果以工程数据的形式验证了 zkSNARK 所要求的 **完整性（Completeness）**与 **可靠性（Soundness）**加密特性。

6.2 实验心得体会

通过本次实验的逐步调试与完成过程，我从中学到了很大的心得体会：

1. **理论与实践的结合**：深入理解了非交互式零知识证明（NIZK）的技术原理，特别是将高级代数方程等价转换为算术电路，并进一步拆解为一阶约束系统（R1CS）的严格数学过程。实验将理论层面的“逻辑约束”实体化为了 `libsnark` 中由 `protoboard`（面包板）承载的 C++ 框架代码，实现了密码学理论的工程实践。

2. **环境配置与依赖构建经验**：在实验初期，我遇到了 libsnark 的 github 源码由于网络限制导致子模块（Git Submodules）拉取失败、CMake 报错找不到底层依赖库等种种问题。但是最终我通过手动离线下载 ZIP 压缩包，并依据 `.gitmodules` 配置文件来完成 `libff`、`gtest` 等子模块的目录存放，最后成功跑通了 libsnark 的底层静态库编译。这极大地锻炼了我通过依靠官方项目架构分析，从而进行排错和构建的逐步动手能力。

综上所述，本次实验不仅让我在理论层面学习验证了零知识证明系统的严密与高效，更在从环境搭建到代码跑通这个整体流程中积累了实践经验。

7 github 仓库

代码详见： https://github.com/Wangxuuuuu/Data-Security/tree/main